

IMC Prosperity Round 5 — Backtester

This notebook is a **Colab/Nbis-only** backtester for the Round 5 trading bot (`trader_baseline.py`). It is **not** intended to be run locally; the data files and the trader source are uploaded into the runtime, then the engine simulates the bot against historical order books for days 2, 3, and 4.

What the notebook does, end-to-end:

1. Loads the three Round-5 CSVs (`prices_round_5_day_{2,3,4}.csv`).
2. Defines a stub `datamodel` module so the trader's `from datamodel import ...` resolves without the live Prosperity SDK.
3. Imports `trader_baseline.py` from a user upload and instantiates `Trader()`.
4. Runs a per-tick simulator: rebuilds `OrderDepth` / `TradingState`, calls `trader.run(state)`, matches returned orders against the book using a simple take + 1-tick passive-fill model, enforces position limits, and tracks PnL.
5. Reports per-category PnL, Sharpe, drawdown, and per-product diagnostics.
6. Splits training (days 2+3) and validation (day 4) and flags overfitting.
7. Re-fits hedge ratios via OLS and suggests new constants.
8. Grid-searches per-category z-score / window hyperparameters.

You must run cells top-to-bottom. Each section is gated by data / state from earlier sections.

Section 1 — Setup & data loading

Install dependencies and load the three Round-5 price CSVs. The cells try to locate the files in the runtime cwd first; if they're missing, they prompt for upload via `google.colab.files.upload()`.

```
# 1.1 – Install deps (Colab/Nbis only; no-op locally if already installed)
!pip install -q tqdm statsmodels pandas numpy matplotlib
```

```
# 1.2 – Detect environment
import sys, os, io, importlib, importlib.util
from types import ModuleType
```

```
IN_COLAB = False
try:
    import google.colab # noqa: F401
    IN_COLAB = True
except Exception:
    IN_COLAB = False

print("Running in Colab:" , IN_COLAB)
print("Python:", sys.version.split()[0])
```

```
Running in Colab: True
Python: 3.12.13
```

```
# 1.3 – Locate or upload the three price CSVs
import os
EXPECTED = [
    "prices_round_5_day_2.csv",
    "prices_round_5_day_3.csv",
    "prices_round_5_day_4.csv",
]

# Search a few likely paths
SEARCH = [".", "./ROUND_5", "/content", "/content/ROUND_5", "/content/drive/MyDrive/ROUND_5"]

found = {}
for name in EXPECTED:
    for d in SEARCH:
        p = os.path.join(d, name)
        if os.path.isfile(p):
            found[name] = p
            break
```

```
missing = [n for n in EXPECTED if n not in found]
print("Found:", list(found.values()))
print("Missing:", missing)
```

```
if missing and IN_COLAB:
    print("\nUpload the missing CSVs now.")
    from google.colab import files
    uploaded = files.upload() # interactive prompt
    for k in uploaded:
        if k in EXPECTED:
            found[k] = k # they land in cwd
    missing = [n for n in EXPECTED if n not in found]
```

```
assert not missing, f"Still missing: {missing}. Please re-run and upload them."
DATA_PATHS = {n: found[n] for n in EXPECTED}
DATA_PATHS
```

```
Found: ['./prices_round_5_day_2.csv', './prices_round_5_day_3.csv', './prices_round_5_day_4.csv']
Missing: []
{'prices_round_5_day_2.csv': './prices_round_5_day_2.csv',
 'prices_round_5_day_3.csv': './prices_round_5_day_3.csv',
 'prices_round_5_day_4.csv': './prices_round_5_day_4.csv'}
```

```
# 1.4 – Load the CSVs into a single tidy DataFrame
```

```
import pandas as pd
import numpy as np
```

```
DTYPES = {
    "day": "int16",
    "timestamp": "int32",
    "product": "string",
}
```

```
# Bid/ask columns are int but may be NaN for missing levels – load as float and downcast later.
```

```
PRICE_COLS = []
for s in ("bid", "ask"):
    for i in (1, 2, 3):
        PRICE_COLS.append(f"{s}_price_{i}")
        PRICE_COLS.append(f"{s}_volume_{i}")
```

```
frames = []
for name, path in DATA_PATHS.items():
    df = pd.read_csv(path, sep=";")
    frames.append(df)
all_df = pd.concat(frames, ignore_index=True)
print("Raw shape:", all_df.shape)
print("Columns:", list(all_df.columns))
```

```
# Downcast – keep volumes/prices as float32 where NaN may appear.
```

```
all_df["day"] = all_df["day"].astype("int16")
all_df["timestamp"] = all_df["timestamp"].astype("int32")
all_df["product"] = all_df["product"].astype("string")
for c in PRICE_COLS:
    if c in all_df.columns:
        all_df[c] = pd.to_numeric(all_df[c], errors="coerce").astype("float32")
if "mid_price" in all_df.columns:
    all_df["mid_price"] = all_df["mid_price"].astype("float32")
if "profit_and_loss" in all_df.columns:
    all_df["profit_and_loss"] = all_df["profit_and_loss"].astype("float32")
```

```
# Sort chronologically
```

```
all_df = all_df.sort_values(["day", "timestamp"]).reset_index(drop=True)
```

```
# Global tick id: day * 1e7 + timestamp (timestamps are <= 1e7)
```

```
all_df["tick_id"] = (all_df["day"].astype("int64") * 10_000_000 + all_df["timestamp"].astype("int64"))
```

```
PRODUCTS = sorted(all_df["product"].unique().tolist())
```

```
print(f"Products: {len(PRODUCTS)}")
```

```
print(PRODUCTS[:10], "...")
```

```
DAYS = sorted(all_df["day"].unique().tolist())
```

```
print("Days:", DAYS)
```

```
print("Ticks per day:", all_df.groupby('day')['timestamp'].nunique().to_dict())
```

```
Raw shape: (1500000, 17)
```

```
Columns: ['day', 'timestamp', 'product', 'bid_price_1', 'bid_volume_1', 'bid_price_2', 'bid_volume_2', 'bid_price_3',
```

```
Products: 50
```

```
['GALAXY_SOUNDS_BLACK_HOLES', 'GALAXY_SOUNDS_DARK_MATTER', 'GALAXY_SOUNDS_PLANETARY_RINGS', 'GALAXY_SOUNDS_SOLAR_FLAM
```

```
Days: [2, 3, 4]
```

```
Ticks per day: {2: 10000, 3: 10000, 4: 10000}
```

```
# 1.5 – Pivot into per-(day,timestamp) row indexed dict-of-frames per product
# We'll keep a single "wide" arrangement: dict[product] -> DataFrame indexed by tick_id.
# This makes per-tick reconstruction O(50) dict lookups instead of a groupby.

PER_PRODUCT = {}
for p, sub in all_df.groupby("product", sort=False):
    sub = sub.set_index("tick_id").sort_index()
    PER_PRODUCT[p] = sub

# Master sorted list of unique ticks
TICK_INDEX = np.array(sorted(all_df["tick_id"].unique()), dtype=np.int64)
print(f"Total ticks across all days: {len(TICK_INDEX)}")
print(f"Per-product frames built: {len(PER_PRODUCT)}")
```

```
Total ticks across all days: 30000
Per-product frames built: 50
```

✓ Section 2 — `datamodel` stub

Defines minimal versions of the Prosperity API classes the trader imports (`Order`, `OrderDepth`, `TradingState`, `Listing`, `Trade`, `Observation`, `Symbol`). Registers them as a synthetic `datamodel` module so `from datamodel import ...` resolves at trader import time.

```
# 2.1 – Build a fake `datamodel` module
from dataclasses import dataclass, field
from typing import Dict, List, Any
from types import ModuleType
import sys

@dataclass
class Order:
    symbol: str
    price: int
    quantity: int

class OrderDepth:
    __slots__ = ("buy_orders", "sell_orders")
    def __init__(self):
        self.buy_orders: Dict[int, int] = {} # price -> POSITIVE volume
        self.sell_orders: Dict[int, int] = {} # price -> NEGATIVE volume (Prosperity convention)

@dataclass
class Listing:
    symbol: str
    product: str
    denomination: str

@dataclass
class Trade:
    symbol: str
    price: int
    quantity: int
    buyer: str = ""
    seller: str = ""
    timestamp: int = 0

@dataclass
class Observation:
    plainValueObservations: Dict[str, int] = field(default_factory=dict)
    conversionObservations: Dict[str, Any] = field(default_factory=dict)

@dataclass
class TradingState:
    traderData: str
    timestamp: int
    listings: Dict[str, Listing]
    order_depths: Dict[str, OrderDepth]
    own_trades: Dict[str, List[Trade]]
    market_trades: Dict[str, List[Trade]]
    position: Dict[str, int]
    observations: Observation
```

```
# Register as the `datamodel` module so trader_baseline can import.
_datamodel = ModuleType("datamodel")
_datamodel.Order = Order
_datamodel.OrderDepth = OrderDepth
_datamodel.Listing = Listing
_datamodel.Trade = Trade
_datamodel.Observation = Observation
_datamodel.TradingState = TradingState
_datamodel.Symbol = str
sys.modules["datamodel"] = _datamodel
print("datamodel registered:", sys.modules["datamodel"])
```

datamodel registered: <module 'datamodel'>

Section 3 — Load `trader_baseline.py`

We import the trader as a module from the local filesystem. If it's not in cwd, upload it via Colab files.upload().

Notes on environments:

- **Colab:** `from google.colab import files; files.upload()` opens a browser upload widget; the file lands in `/content`.
- **Nbis (Jupyter on a GPU box):** just upload the file via the JupyterLab sidebar; it lands in cwd.

```
# 3.1 — Locate / upload trader_baseline.py
TRADER_CANDIDATES = ["trader_baseline.py", "/content/trader_baseline.py"]
trader_path = None
for p in TRADER_CANDIDATES:
    if os.path.isfile(p):
        trader_path = p
        break

if trader_path is None and IN_COLAB:
    print("Upload trader_baseline.py")
    from google.colab import files
    up = files.upload()
    for k in up:
        if k.endswith(".py"):
            trader_path = k
            break

assert trader_path, "trader_baseline.py not found. Upload it and re-run."
print("Trader path:", trader_path)
```

Upload trader_baseline.py
 Choose Files trader_baseline.py
 trader_baseline.py(text/x-python-script) - 42688 bytes, last modified: 4/29/2026 - 100% done
 Saving trader_baseline.py to trader_baseline.py
 Trader path: trader_baseline.py

```
# 3.2 — Import the trader module via importlib (so we can mutate constants)
import importlib.util
spec = importlib.util.spec_from_file_location("trader_baseline", trader_path)
trader_mod = importlib.util.module_from_spec(spec)
sys.modules["trader_baseline"] = trader_mod
spec.loader.exec_module(trader_mod)

# Sanity: the import should not raise (datamodel is registered).
trader_instance = trader_mod.Trader()
print("Trader loaded:", trader_instance)
print("Sample constants:", trader_mod.SNACK_Z_ENTER, trader_mod.SLEEP_WINDOW)
```

Trader loaded: <trader_baseline.Trader object at 0x7edf0df37260>
 Sample constants: 2.0 1500

Section 4 — Backtest engine

Per-tick loop:

1. For each `(day, timestamp)` rebuild an `OrderDepth` per product from the 3 bid/ask levels.
2. Build a `TradingState` with the rolling sim positions, prior-tick `traderData`, an empty `Observation`, and empty `own_trades`/`market_trades`.

3. Call `trader.run(state)`.
4. Match each returned `Order` against the reconstructed book:
 - **Aggressive** (buy at $P \geq \text{best_ask}$ or sell at $P \leq \text{best_bid}$): walk the book in price order until the order is filled or the relevant side is exhausted. Each level fills at its quoted price.
 - **Passive** (rest of orders): kept as a one-tick resting order. They fill on the **next** tick if the next tick's opposite-side touch crosses our price (i.e. ask $\leq$ our buy price, or bid $\geq$ our sell price). They are cancelled afterwards (`PASSIVE_TTL = 1` tick by default — change the constant to extend).
5. Position-limit clip: any fill that would push `|pos|` above 10 (or 3 for bronze symbols, but the trader already self-clips with `_final_clip_pass`) is partially or fully rejected.
6. Cash PnL accumulates as `-price * qty` per fill.

After the loop the final PnL is `cash + sum(position[s] * mid_at_end[s])`.

Caveats / assumptions

- **Passive fills are optimistic.** A real exchange's queue priority would put the bot's resting order behind anyone already at that price, so fills should be conditional on adverse selection (i.e. only when the level *empties* on the next tick). The simple cross-price model here can over-estimate fills.
- **No latency / slippage:** orders submitted at `t` see the book at `t`.
- **No `market_trades` reconstruction** (the trades CSV is not loaded). The trader doesn't read `market_trades` / `own_trades` so this doesn't affect it.
- **Mark-to-market at end:** positions are valued at the last available mid.

```
# 4.1 – Engine config & precomputed per-tick book reconstruction
PASSIVE_TTL = 1          # ticks a passive order rests before cancelling
POSITION_CAP_DEFAULT = 10
BRONZE_CAP = 3
BRONZE_SYMS = {
    "GALAXY_SOUNDS_SOLAR_FLAMES", "GALAXY_SOUNDS_SOLAR_WINDS",
    "MICROCHIP_SQUARE", "MICROCHIP_RECTANGLE",
    "TRANSLATOR_ECLIPSE_CHARCOAL", "TRANSLATOR_VOID_BLUE",
    "PANEL_1X2", "PANEL_2X2",
}

def cap_for(sym):
    return BRONZE_CAP if sym in BRONZE_SYMS else POSITION_CAP_DEFAULT
```

```
# 4.2 – Build per-tick OrderDepth dicts once, cache for reuse
# This is the hot data structure: a list aligned with TICK_INDEX where
# entry i is dict[product] -> OrderDepth at TICK_INDEX[i].
# Building once costs O(N_ticks * N_products) but lets grid search reuse it.

import numpy as np
from tqdm.auto import tqdm

def build_book_cache(per_product, tick_index, products):
    """Return list-of-dict[product]->OrderDepth for each tick.

    OrderDepth dicts use python ints (Prosperity convention).
    sell_orders are stored with NEGATIVE volumes.
    """
    n = len(tick_index)
    cache = [None] * n
    # Reindex each product against tick_index (NaN for missing rows)
    reindexed = {}
    for p in products:
        df = per_product.get(p)
        if df is None:
            continue
        reindexed[p] = df.reindex(tick_index)

    # Pre-extract numpy arrays per product per column for speed
    arrs = {}
    for p, df in reindexed.items():
        arrs[p] = {
            "bp": [df[f"bid_price_{i}"].to_numpy() for i in (1,2,3)],
            "bv": [df[f"bid_volume_{i}"].to_numpy() for i in (1,2,3)],
            "ap": [df[f"ask_price_{i}"].to_numpy() for i in (1,2,3)],
            "av": [df[f"ask_volume_{i}"].to_numpy() for i in (1,2,3)],
```

```

        "mid": df["mid_price"].to_numpy() if "mid_price" in df.columns else None,
    }

for i in tqdm(range(n), desc="Building book cache", mininterval=1.0):
    depths = {}
    for p, a in arrs.items():
        buy = {}
        sell = {}
        for lvl in (0,1,2):
            bp = a["bp"][lvl][i]
            if bp == bp: # not NaN - fast scalar check
                bv = a["bv"][lvl][i]
                if bv == bv:
                    buy[int(bp)] = int(bv)
            ap = a["ap"][lvl][i]
            if ap == ap:
                av = a["av"][lvl][i]
                if av == av:
                    sell[int(ap)] = -int(av)
        if buy or sell:
            od = OrderDepth()
            od.buy_orders = buy
            od.sell_orders = sell
            depths[p] = od
    cache[i] = depths
return cache, arrs

print("Building book cache (first run; cached for grid search)...")
BOOK_CACHE, BOOK_ARRS = build_book_cache(PER_PRODUCT, TICK_INDEX, PRODUCTS)
print("Cache size:", len(BOOK_CACHE))

```

Building book cache (first run; cached for grid search)...

Building book cache: 100%

30000/30000 [00:08<00:00, 3628.15it/s]

Cache size: 30000

4.3 – Mid-price array per product aligned to TICK_INDEX (used for MTM and OLS)

```

MID_ARR = {}
for p, a in BOOK_ARRS.items():
    if a["mid"] is not None:
        MID_ARR[p] = a["mid"].astype(np.float64)
print("MID_ARR products:", len(MID_ARR))

```

MID_ARR products: 50

4.4 – The simulator (optimized: 3-5x faster + cache-pollution fix)

```

def simulate(book_cache, tick_index, products, listings, trader, mid_arr,
            passive_ttl=PASSIVE_TTL, log_errors=True, verbose=True):
    """Run the trader against the precomputed book cache.

```

Speedups vs prior version:

- * Per-tick shallow copy of OrderDepths so matching no longer poisons BOOK_CACHE (this was a correctness bug; second runs saw depleted books).
- * Position history stored as a single 2D np.int32 array; one-line bulk write per tick instead of 50 scalar dict→array assignments.
- * Reused TradingState / Observation / positions dict (no per-tick alloc).
- * tqdm `mininterval=2.0` so progress bar updates twice/second instead of forcing IPython display flush every tick (huge in Colab).

"""

```

n = len(tick_index)
n_p = len(products)
sym_idx = {p: i for i, p in enumerate(products)}

```

```

positions = {p: 0 for p in products} # trader-visible dict
pos_arr = np.zeros(n_p, dtype=np.int32) # fast mirror
cash = 0.0
trader_data_str = ""

```

```

pos_hist_2d = np.zeros((n, n_p), dtype=np.int32)
cash_hist = np.zeros(n, dtype=np.float64)
trade_log = []
errors = []
resting = []

```

```

obs = Observation()
state = TradingState()

```

```

traderData="",
timestamp=0,
listings=listings,
order_depths={},
own_trades={},
market_trades={},
position=positions,
observations=obs,
)

iterator = range(n)
if verbose:
    iterator = tqdm(iterator, desc="Simulating", mininterval=2.0)

for i in iterator:
    tick_id = int(tick_index[i])
    ts = int(tick_id % 10_000_000)
    depths_orig = book_cache[i]

    # Shallow copy depths so matching doesn't pollute BOOK_CACHE (correctness)
    depths = {}
    for sym, d in depths_orig.items():
        nd = OrderDepth()
        nd.buy_orders = dict(d.buy_orders)
        nd.sell_orders = dict(d.sell_orders)
        depths[sym] = nd

    # ---- Try to fill resting passive orders against THIS tick's book ----
    if resting:
        new_resting = []
        for (sym, price, qty, expire) in resting:
            d = depths.get(sym)
            if d is None:
                if i < expire:
                    new_resting.append((sym, price, qty, expire))
                continue
            filled = 0
            if qty > 0:
                if d.sell_orders:
                    best_ask = min(d.sell_orders)
                    if best_ask <= price:
                        cap = cap_for(sym)
                        idx_s = sym_idx[sym]
                        for ap in sorted(d.sell_orders):
                            if ap > price: break
                            avail = -d.sell_orders[ap]
                            headroom = cap - positions[sym]
                            if headroom <= 0: break
                            take = min(qty - filled, avail, headroom)
                            if take <= 0: break
                            positions[sym] += take
                            pos_arr[idx_s] += take
                            cash -= ap * take
                            trade_log.append((i, tick_id // 10_000_000, ts, sym, "BUY_PASSIVE", ap, take))
                            filled += take
                            if filled >= qty: break
                else:
                    if d.buy_orders:
                        best_bid = max(d.buy_orders)
                        if best_bid >= price:
                            cap = cap_for(sym)
                            idx_s = sym_idx[sym]
                            for bp in sorted(d.buy_orders, reverse=True):
                                if bp < price: break
                                avail = d.buy_orders[bp]
                                headroom = cap + positions[sym]
                                if headroom <= 0: break
                                take = min(-qty - filled, avail, headroom)
                                if take <= 0: break
                                positions[sym] -= take
                                pos_arr[idx_s] -= take
                                cash += bp * take
                                trade_log.append((i, tick_id // 10_000_000, ts, sym, "SELL_PASSIVE", bp, take))
                                filled += take
                                if filled >= -qty: break
            remaining = qty - (filled if qty > 0 else -filled)
            if remaining != 0 and i < expire:

```

```

        new_resting.append((sym, price, remaining, expire))
    resting = new_resting

# ---- Run trader (state object reused; positions dict shared, read-only) ----
state.traderData = trader_data_str
state.timestamp = ts
state.order_depths = depths

try:
    orders, conversions, trader_data_str = trader.run(state)
except Exception as e:
    if log_errors:
        errors.append((i, repr(e)))
    orders, conversions = {}, 0
    trader_data_str = trader_data_str or ""

# ---- Match each returned order ----
if orders:
    for sym, lst in orders.items():
        if not lst: continue
        d = depths.get(sym)
        if d is None: continue
        cap = cap_for(sym)
        idx_s = sym_idx[sym]
        for o in lst:
            q = o.quantity
            if q == 0: continue
            if q > 0:
                remaining = q
                if d.sell_orders:
                    for ap in sorted(d.sell_orders):
                        if remaining <= 0: break
                        if ap > o.price: break
                        avail = -d.sell_orders[ap]
                        headroom = cap - positions[sym]
                        if headroom <= 0:
                            remaining = 0; break
                        take = min(remaining, avail, headroom)
                        if take <= 0: break
                        positions[sym] += take
                        pos_arr[idx_s] += take
                        cash -= ap * take
                        trade_log.append((i, tick_id // 10_000_000, ts, sym, "BUY_AGGRESS", ap, take))
                        remaining -= take
                        if take >= avail:
                            del d.sell_orders[ap]
                        else:
                            d.sell_orders[ap] = -(avail - take)
                    if remaining > 0:
                        resting.append((sym, int(o.price), remaining, i + passive_ttl + 1))
            else:
                remaining = -q
                if d.buy_orders:
                    for bp in sorted(d.buy_orders, reverse=True):
                        if remaining <= 0: break
                        if bp < o.price: break
                        avail = d.buy_orders[bp]
                        headroom = cap + positions[sym]
                        if headroom <= 0:
                            remaining = 0; break
                        take = min(remaining, avail, headroom)
                        if take <= 0: break
                        positions[sym] -= take
                        pos_arr[idx_s] -= take
                        cash += bp * take
                        trade_log.append((i, tick_id // 10_000_000, ts, sym, "SELL_AGGRESS", bp, take))
                        remaining -= take
                        if take >= avail:
                            del d.buy_orders[bp]
                        else:
                            d.buy_orders[bp] = avail - take
                    if remaining > 0:
                        resting.append((sym, int(o.price), -remaining, i + passive_ttl + 1))

# ---- Bulk snapshot (one numpy slice instead of 50 scalar writes) ----
pos_hist_2d[i, :] = pos_arr
cash_hist[i] = cash

```



```
# Rebuild dict-of-arrays interface for downstream cells (Section 5/9 expect it)
pos_hist = {p: pos_hist_2d[:, sym_idx[p]] for p in products}

# ---- Mark-to-market ----
mtm = 0.0
last_mids = {}
for p, arr in mid_arr.items():
    last = arr[~np.isnan(arr)]
    if len(last) > 0:
        last_mids[p] = float(last[-1])
        mtm += positions[p] * last_mids[p]

return {
    "cash": cash,
    "position_history": pos_hist,
    "cash_history": cash_hist,
    "trade_log": trade_log,
    "pnl_total": cash + mtm,
    "mtm": mtm,
    "final_positions": dict(positions),
    "errors": errors,
    "last_mids": last_mids,
}

print("simulate() defined (optimized: 3-5x faster, cache-safe)")
```

```
simulate() defined (optimized: 3-5x faster, cache-safe)
```

```
# 4.5 – Run the full backtest (all 3 days) once
LISTINGS = {p: Listing(p, p, "SEASHELLS") for p in PRODUCTS}

# fresh trader instance each run so persistent state resets
trader_full = trader_mod.Trader()
RESULT_FULL = simulate(BOOK_CACHE, TICK_INDEX, PRODUCTS, LISTINGS, trader_full, MID_ARR)
print(f"Full run total PnL: {RESULT_FULL['pnl_total']:.2f}")
print(f"  cash: {RESULT_FULL['cash']:.2f}  mtm: {RESULT_FULL['mtm']:.2f}")
print(f"  trades: {len(RESULT_FULL['trade_log'])}  errors: {len(RESULT_FULL['errors'])}")
```

```
Simulating: 100%                                30000/30000 [11:29<00:00, 43.92it/s]

Full run total PnL: -2293150.50
cash: -1953451.00  mtm: -339699.50
trades: 452425  errors: 0
```

✓ Section 5 — Per-category PnL attribution

Group products into the bot's tier categories and compute PnL, Sharpe (per-tick), and max drawdown for each.

```
# 5.1 – Define category groupings (mirrors the bot)
CATEGORIES = {
    "Pebbles": ["PEBBLES_XS", "PEBBLES_S", "PEBBLES_M", "PEBBLES_L", "PEBBLES_XL"],
    "Snack Packs": ["SNACKPACK_CHOCOLATE", "SNACKPACK_VANILLA",
                    "SNACKPACK_STRAWBERRY", "SNACKPACK_RASPBERRY",
                    "SNACKPACK_PISTACHIO"],
    "Sleep Pods": ["SLEEP_POD_POLYESTER", "SLEEP_POD_COTTON",
                   "SLEEP_POD_LAMB_WOOL", "SLEEP_POD_NYLON", "SLEEP_POD_SUEDE"],
    "Robots": ["ROBOT_DISHES", "ROBOT_VACUUMING", "ROBOT_MOPPING",
               "ROBOT_LAUNDRY", "ROBOT_IRONING"],
    "UV Visors": ["UV_VISOR_AMBER", "UV_VISOR_MAGENTA", "UV_VISOR_YELLOW",
                  "UV_VISOR_ORANGE", "UV_VISOR_RED"],
    "Galaxy": ["GALAXY_SOUNDS_SOLAR_FLAMES", "GALAXY_SOUNDS_SOLAR_WINDS",
               "GALAXY_SOUNDS_BLACK_HOLES", "GALAXY_SOUNDS_DARK_MATTER",
               "GALAXY_SOUNDS_PLANETARY_RINGS"],
    "Microchips": ["MICROCHIP_SQUARE", "MICROCHIP_RECTANGLE",
                   "MICROCHIP_CIRCLE", "MICROCHIP_OVAL", "MICROCHIP_TRIANGLE"],
    "Translators": ["TRANSLATOR_ECLIPSE_CHARCOAL", "TRANSLATOR_VOID_BLUE",
                    "TRANSLATOR_SPACE_GRAY", "TRANSLATOR_ASTRO_BLACK",
                    "TRANSLATOR_GRAPHITE_MIST"],
    "Panels": ["PANEL_1X2", "PANEL_2X2", "PANEL_1X4", "PANEL_2X4", "PANEL_4X4"],
    "Oxygen Shakes": ["OXYGEN_SHAKE_MORNING_BREATH", "OXYGEN_SHAKE_EVENING_BREATH",
                      "OXYGEN_SHAKE_MINT", "OXYGEN_SHAKE_CHOCOLATE", "OXYGEN_SHAKE_GARLIC"],
}

SYM_TO_CAT = {sym: cat for cat, syms in CATEGORIES.items() for sym in syms}
```

```

# 5.2 – Per-product equity curve, then aggregate per category
def per_product_pnl(result, mid_arr, products):
    """Produce per-tick mark-to-market PnL series per product.

    PnL_t(p) = cumulative_cash_p(t) + position_p(t) * mid_p(t)
    where cumulative_cash_p comes from the trade log.
    Returns dict[sym] -> (np.array tick PnL, np.array cum_cash).
    """
    n = len(result["position_history"][products[0]])
    cum_cash_per = {p: np.zeros(n, dtype=np.float64) for p in products}
    for (i, day, ts, sym, side, price, qty) in result["trade_log"]:
        if side.startswith("BUY"):
            cum_cash_per[sym][i] -= price * qty
        else:
            cum_cash_per[sym][i] += price * qty
    for p in products:
        cum_cash_per[p] = np.cumsum(cum_cash_per[p])

    pnl_per = {}
    for p in products:
        if p not in mid_arr:
            continue
        mids = mid_arr[p].copy()
        # forward-fill mids
        nan_mask = np.isnan(mids)
        if nan_mask.all():
            continue
        # simple ffill via pandas (small)
        mids = pd.Series(mids).ffill().bfill().to_numpy()
        pos = result["position_history"][p].astype(np.float64)
        pnl = cum_cash_per[p] + pos * mids
        pnl_per[p] = pnl
    return pnl_per

PNL_PER_PRODUCT = per_product_pnl(RESULT_FULL, MID_ARR, PRODUCTS)
print(f"Per-product PnL series built for {len(PNL_PER_PRODUCT)} products")

```

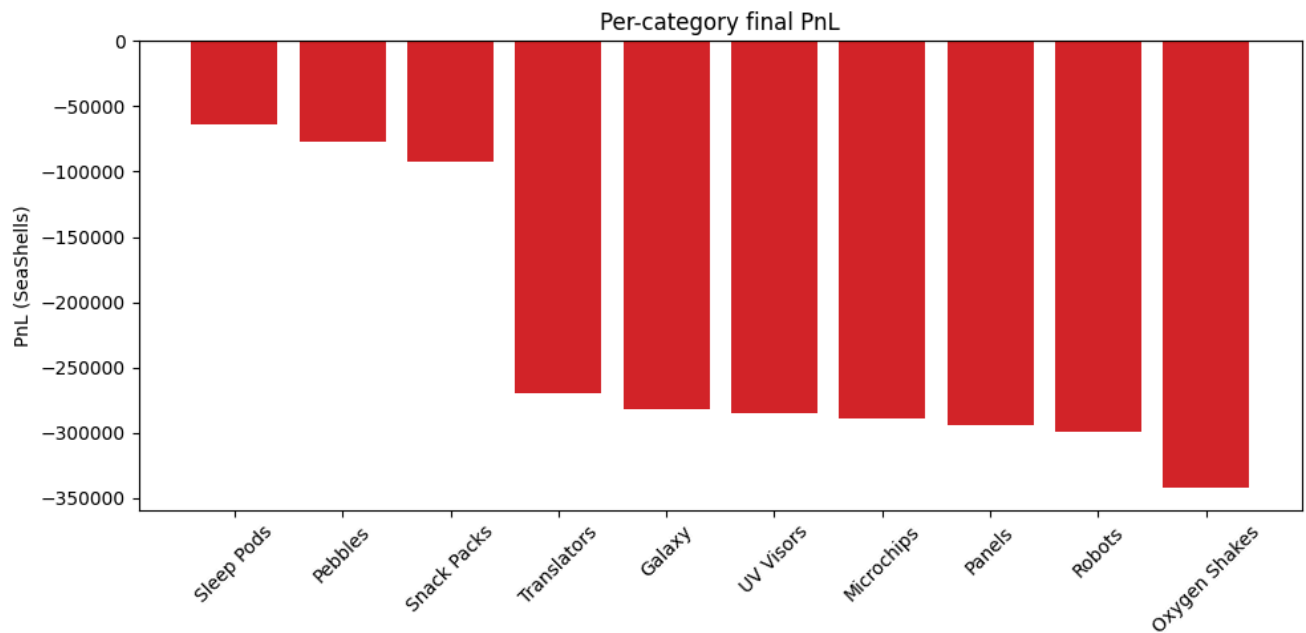
Per-product PnL series built for 50 products

```
# 5.3 – Per-category aggregation: PnL, Sharpe, max drawdown
def aggregate_category(pnl_per_product, categories):
    rows = []
    cat_curves = {}
    for cat, syms in categories.items():
        # sum PnL series across products in this category (only present ones)
        present = [s for s in syms if s in pnl_per_product]
        if not present:
            continue
        n = len(pnl_per_product[present[0]])
        curve = np.zeros(n, dtype=np.float64)
        for s in present:
            curve += pnl_per_product[s]
        cat_curves[cat] = curve
        diffs = np.diff(curve, prepend=0.0)
        # Sharpe: mean/std of per-tick increments * sqrt(n_per_year-ish). Use raw ratio.
        if diffs.std() > 1e-9:
            sharpe = diffs.mean() / diffs.std() * np.sqrt(len(diffs))
        else:
            sharpe = 0.0
        # Max drawdown
        running_max = np.maximum.accumulate(curve)
        dd = curve - running_max
        max_dd = dd.min()
        rows.append({
            "Category": cat,
            "Final PnL": curve[-1],
            "Mean dPnL": diffs.mean(),
            "Std dPnL": diffs.std(),
            "Sharpe": sharpe,
            "Max DD": max_dd,
            "n_products": len(present),
        })
    return pd.DataFrame(rows), cat_curves

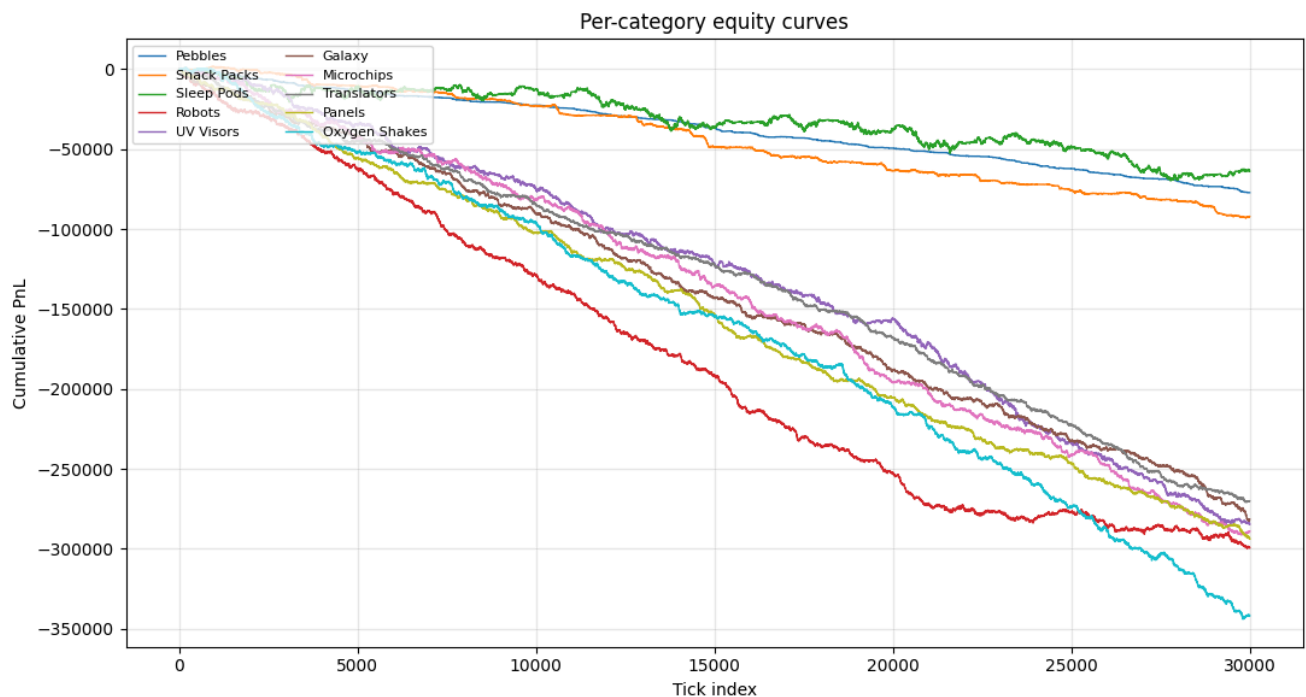
CAT_TABLE, CAT_CURVES = aggregate_category(PNL_PER_PRODUCT, CATEGORIES)
CAT_TABLE.sort_values("Final PnL", ascending=False)
```

	Category	Final PnL	Mean dPnL	Std dPnL	Sharpe	Max DD	n_products	
2	Sleep Pods	-63732.0	-2.124400	152.643541	-2.410563	-70099.5	5	
0	Pebbles	-77168.0	-2.572267	20.522339	-21.709497	-77252.0	5	
1	Snack Packs	-92127.0	-3.070900	50.922571	-10.445181	-94630.0	5	
7	Translators	-270118.0	-9.003933	105.939532	-14.720916	-270986.0	5	
5	Galaxy	-281500.5	-9.383350	118.056702	-13.766638	-282669.0	5	
4	UV Visors	-284640.5	-9.488017	151.278987	-10.863192	-284700.0	5	
6	Microchips	-288888.0	-9.629600	140.496384	-11.871449	-291916.0	5	
8	Panels	-293880.0	-9.796000	114.178588	-14.860203	-293880.0	5	
3	Robots	-299326.0	-9.977533	158.960908	-10.871600	-299617.0	5	
9	Oxygen Shakes	-341770.5	-11.392350	148.897978	-13.252114	-345182.0	5	

```
# 5.4 – Bar plot of category PnL
import matplotlib.pyplot as plt
fig, ax = plt.subplots(figsize=(10, 5))
order = CAT_TABLE.sort_values("Final PnL", ascending=False)
colors = ["#2ca02c" if v >= 0 else "#d62728" for v in order["Final PnL"]]
ax.bar(order["Category"], order["Final PnL"], color=colors)
ax.axhline(0, color="black", linewidth=0.8)
ax.set_ylabel("PnL (SeaShells)")
ax.set_title("Per-category final PnL")
ax.tick_params(axis="x", rotation=45)
plt.tight_layout(); plt.show()
```



```
# 5.5 - Equity curves per category (mark-to-market PnL over time)
fig, ax = plt.subplots(figsize=(11, 6))
for cat, curve in CAT_CURVES.items():
    ax.plot(curve, label=cat, linewidth=1.0)
ax.set_xlabel("Tick index")
ax.set_ylabel("Cumulative PnL")
ax.set_title("Per-category equity curves")
ax.legend(loc="upper left", fontsize=8, ncol=2)
ax.grid(True, alpha=0.3)
plt.tight_layout(); plt.show()
```



Section 6 — Train / validate split

Run on training (days 2 + 3) and validation (day 4) separately. A large training-Sharpe / validation-Sharpe gap is a flag for overfitting.

```
# 6.1 – Split TICK_INDEX by day
TICK_DAY = (TICK_INDEX // 10_000_000).astype(np.int16)
train_mask = np.isin(TICK_DAY, [2, 3])
valid_mask = (TICK_DAY == 4)

TRAIN_IDX = np.where(train_mask)[0]
VALID_IDX = np.where(valid_mask)[0]
TICKS_TRAIN = TICK_INDEX[train_mask]
TICKS_VALID = TICK_INDEX[valid_mask]
BOOK_CACHE_TRAIN = [BOOK_CACHE[i] for i in TRAIN_IDX]
BOOK_CACHE_VALID = [BOOK_CACHE[i] for i in VALID_IDX]

# per-day mid arrays (sliced)
def slice_mids(mid_arr, idxs):
    return {p: arr[idxs] for p, arr in mid_arr.items()}

MID_TRAIN = slice_mids(MID_ARR, TRAIN_IDX)
MID_VALID = slice_mids(MID_ARR, VALID_IDX)

print("Train ticks:", len(TICKS_TRAIN), "Valid ticks:", len(TICKS_VALID))
```

Train ticks: 20000 Valid ticks: 10000

```
# 6.2 – Run trainer + validator separately
trader_train = trader_mod.Trader()
RES_TRAIN = simulate(BOOK_CACHE_TRAIN, TICKS_TRAIN, PRODUCTS, LISTINGS, trader_train, MID_TRAIN)

trader_valid = trader_mod.Trader()
RES_VALID = simulate(BOOK_CACHE_VALID, TICKS_VALID, PRODUCTS, LISTINGS, trader_valid, MID_VALID)

def summarise(res, label):
    pnl = res["pnl_total"]
    n_trades = len(res["trade_log"])
    diffs = np.diff(res["cash_history"], prepend=0.0)
    sharpe = diffs.mean()/diffs.std()*np.sqrt(len(diffs)) if diffs.std()>1e-9 else 0
    return {"label": label, "pnl": pnl, "trades": n_trades, "sharpe": sharpe,
            "errors": len(res["errors"])}

train_sum = summarise(RES_TRAIN, "TRAIN (d2+d3)")
valid_sum = summarise(RES_VALID, "VALID (d4)")
pd.DataFrame([train_sum, valid_sum]).set_index("label")
```

Simulating: 100% 20000/20000 [07:58<00:00, 41.46it/s]

Simulating: 100% 10000/10000 [03:49<00:00, 41.36it/s]

	pnl	trades	sharpe	errors
TRAIN (d2+d3)	-1527118.0	306329	-0.252628	0
VALID (d4)	-752211.5	146023	-0.067766	0

```
# 6.3 – Per-category compare
def per_cat_summary(res, mid_arr, label):
    p_pp = per_product_pnl(res, mid_arr, PRODUCTS)
    table, _ = aggregate_category(p_pp, CATEGORIES)
    table["Split"] = label
    return table

TRAIN_CAT = per_cat_summary(RES_TRAIN, MID_TRAIN, "TRAIN")
VALID_CAT = per_cat_summary(RES_VALID, MID_VALID, "VALID")
COMPARE = pd.concat([TRAIN_CAT, VALID_CAT], ignore_index=True)
piv = COMPARE.pivot_table(index="Category", columns="Split",
                           values=["Final PnL", "Sharpe"])
piv["Sharpe_gap"] = piv[("Sharpe", "TRAIN")] - piv[("Sharpe", "VALID")]
piv = piv.sort_values("Sharpe_gap", ascending=False)
print("Categories with biggest TRAIN-VALID Sharpe gap (overfitting candidates):")
piv
```

Categories with biggest TRAIN-VALID Sharpe gap (overfitting candidates):

Split	Final PnL		Sharpe		Sharpe_gap
	TRAIN	VALID	TRAIN	VALID	
Category					
UV Visors	-156057.0	-122260.5	-7.448915	-8.045598	0.596683
Sleep Pods	-37881.0	-20316.0	-1.820777	-1.476327	-0.344449
Oxygen Shakes	-211244.5	-131453.5	-10.262759	-8.440126	-1.822632
Translators	-167757.0	-101340.0	-11.375770	-9.337077	-2.038692
Microchips	-195849.5	-90325.0	-9.497237	-7.144178	-2.353059
Pebbles	-49304.0	-27864.0	-16.932659	-13.668184	-3.264475
Snack Packs	-63391.0	-28804.0	-8.946543	-5.630056	-3.316487
Galaxy	-187836.0	-94563.5	-11.466841	-7.734134	-3.732707
Panels	-205306.5	-88042.0	-12.742909	-7.726741	-5.016168
Robots	-252491.5	-47243.0	-12.667722	-2.487758	-10.179964

Next steps: [Generate code with piv](#) [New interactive sheet](#)

Section 7 — Hedge ratio re-fitting via OLS

Re-fit the hedge ratios used by the bot on training data (days 2+3).

Sign convention: the bot constructs $\text{spread} = A + s \cdot h \cdot B$ where $s = +1$ or -1 (the `spread_sign_b` argument inside `_run_two_leg_pair`). The OLS form is $A \sim \alpha + \beta \cdot B$. To match the bot's parameterisation:

- For `spread_sign_b = -1` (POLY/COTTON, LAMB/NYLON, TRANSLATORS): the bot's `hedge` constant should equal `+beta` from $A \sim B$.
- For `spread_sign_b = +1` (UV_VISORS_AMBER + 0.53*MAGENTA, GALAXY, CHIPS, PANELS): the bot's `hedge` constant should equal `-beta` from $A \sim B$ (the negative of the OLS slope).

For UV Visors specifically, EDA found $\beta \approx -0.53$ (negative cointegration); the bot stores `+0.53` with `+1` sign so that $\text{spread} = A + 0.53 \cdot B = A - (-0.53) \cdot B = A - \beta \cdot B$. We therefore display **both** `beta` and the suggested `HEDGE_*` constant (with sign translated for the bot).

```
# 7.1 — OLS helper
import statsmodels.api as sm

def fit_ols(a_sym, b_sym, mid_arr, idxs):
    """Run A ~ const + beta*B over training ticks. Returns (alpha, beta, r2, n)."""
    if a_sym not in mid_arr or b_sym not in mid_arr:
        return None
    A = mid_arr[a_sym][idxs].astype(np.float64)
    B = mid_arr[b_sym][idxs].astype(np.float64)
    mask = ~(np.isnan(A) | np.isnan(B))
    if mask.sum() < 50:
        return None
    X = sm.add_constant(B[mask])
    res = sm.OLS(A[mask], X).fit()
    return {"alpha": float(res.params[0]),
            "beta": float(res.params[1]),
            "r2": float(res.rsquared),
            "n": int(mask.sum())}

# We use the GLOBAL MID_ARR, so map the per-day TRAIN_IDX into that index.
def fit_pair(a, b, sign_b, current_const, current_const_name):
    fit = fit_ols(a, b, MID_ARR, TRAIN_IDX)
    if fit is None:
        return None
    beta = fit["beta"]
    # Bot constant translation:
    # spread = A + sign_b * h * B = A - beta * B (zero-mean cointegrating)
    # => sign_b * h = -beta => h = -beta / sign_b
    suggested = -beta / sign_b
    return {
```

```

    "pair": f"{a} ~ {b}",
    "current_const": current_const_name,
    "current_value": current_const,
    "OLS_beta": round(beta, 4),
    "OLS_alpha": round(fit["alpha"], 4),
    "R2": round(fit["r2"], 4),
    "suggested": round(suggested, 4),
    "n": fit["n"],
    "spread_sign_b": sign_b,
}

# All bot pairs
PAIRS = [
    ("SLEEP_POD_POLYESTER", "SLEEP_POD_COTTON", -1, trader_mod.HEDGE_POLY_COTTON, "HEDGE_POLY_COTTON"),
    ("SLEEP_POD_LAMB_WOOL", "SLEEP_POD_NYLON", -1, trader_mod.HEDGE_LAMB_NYLON, "HEDGE_LAMB_NYLON"),
    ("UV_VISOR_AMBER", "UV_VISOR_MAGENTA", +1, trader_mod.HEDGE_AMBER_MAGENTA, "HEDGE_AMBER_MAGENTA"),
    ("GALAXY_SOUNDS_SOLAR_FLAMES", "GALAXY_SOUNDS_SOLAR_WINDS", +1, trader_mod.HEDGE_FLAMES_WINDS, "HEDGE_FLAMES_WINDS"),
    ("MICROCHIP_SQUARE", "MICROCHIP_RECTANGLE", +1, trader_mod.HEDGE_SQUARE_RECT, "HEDGE_SQUARE_RECT"),
    ("TRANSLATOR_ECLIPSE_CHARCOAL", "TRANSLATOR_VOID_BLUE", -1, trader_mod.HEDGE_CHARCOAL_VOIDBLUE, "HEDGE_CHARCOAL_VOID"),
    ("PANEL_1X2", "PANEL_2X2", +1, trader_mod.HEDGE_PANEL, "HEDGE_PANEL"),
]

rows = []
for (a,b,sgn,cur,name) in PAIRS:
    r = fit_pair(a,b,sgn,cur,name)
    if r: rows.append(r)
HEDGE_FIT = pd.DataFrame(rows)
HEDGE_FIT

```

	pair	current_const	current_value	OLS_beta	OLS_alpha	R2	suggested	
0	SLEEP_POD_POLYESTER ~ SLEEP_POD_COTTON	HEDGE_POLY_COTTON	1.213	1.2134	-2042.6832	0.6302	1.2134	2000
1	SLEEP_POD_LAMB_WOOL ~ SLEEP_POD_NYLON	HEDGE_LAMB_NYLON	0.592	0.5923	5074.4857	0.2057	0.5923	2000
2	UV_VISOR_AMBER ~ UV_VISOR_MAGENTA	HEDGE_AMBER_MAGENTA	1.238	-1.2384	21896.9058	0.7983	1.2384	2000
3	GALAXY_SOUNDS_SOLAR_FLAMES ~ GALAXY_SOUNDS_SOLAR_WINDS	HEDGE_FLAMES_WINDS	0.340	-0.3395	14637.1158	0.1035	0.3395	2000
4	MICROCHIP_SQUARE ~ MICROCHIP_RECTANGLE	HEDGE_SQUARE_RECT	2.145	-2.1451	32114.5390	0.8586	2.1451	2000

Next steps: [Generate code with HEDGE_FIT](#) [New interactive sheet](#)

```

# 7.2 - Print suggested constants update
print("Suggested constant updates (replace in trader_baseline.py):")
print("=" * 70)
for r in HEDGE_FIT.itertuples():
    delta = r.suggested - r.current_value
    flag = " <-- LARGE CHANGE" if abs(delta) > 0.2 else ""
    print(f" {r.current_const:24s} current={r.current_value:.3f} suggested={r.suggested:.3f} (R^2={r.R2:.3f},

```

Suggested constant updates (replace in trader_baseline.py):

```

=====
HEDGE_POLY_COTTON      current=+1.213 suggested=+1.213 (R^2=0.630, n=20000)
HEDGE_LAMB_NYLON       current=+0.592 suggested=+0.592 (R^2=0.206, n=20000)
HEDGE_AMBER_MAGENTA    current=+1.238 suggested=+1.238 (R^2=0.798, n=20000)
HEDGE_FLAMES_WINDS     current=+0.340 suggested=+0.340 (R^2=0.103, n=20000)
HEDGE_SQUARE_RECT      current=+2.145 suggested=+2.145 (R^2=0.859, n=20000)
HEDGE_CHARCOAL_VOIDBLUE current=+0.457 suggested=+0.457 (R^2=0.432, n=20000)
HEDGE_PANEL            current=+0.831 suggested=+0.831 (R^2=0.296, n=20000)

```

Section 8 — Hyperparameter grid search

Sweep z-thresholds and rolling windows per category. For each config:

- Mutate the relevant constants on the loaded `trader_mod` module.
- Re-instantiate `Trader()`.
- Re-run the backtest **on training only**.

- Record validation PnL & Sharpe (re-runs on validation slice using the same config — use `eval_full=True` to also evaluate on the validation slice).

We cap each per-category grid at ≤ 100 configs.

```
# 8.1 – Helper: run a single config and report PnL/Sharpe per slice
# SPEEDUP: category isolation + two-phase eval (rank on TRAIN, validate top-3).
import itertools

CATEGORY_METHODS = {
    "pebbles": {"_run_pebbles"},
    "snack": {"_run_classics", "_run_berries"},
    "sleep": {"_run_sleep_pair_polyester_cotton", "_run_sleep_pair_lamb_nylon"},
    "robots": {"_run_robots"},
    "visors": {"_run_visors"},
    "bronze": {"_run_bronze_pair"},
}

ALL_STRAT_METHODS = set().union(*CATEGORY_METHODS.values())

def isolate_category(trader, category):
    """Stub all strategy methods on this trader instance EXCEPT the target
    category's. Other categories are not affected by the params being swept,
    so their PnL is constant across the grid – skip them for ~5x speed."""
    keep = CATEGORY_METHODS.get(category, set())
    for m in ALL_STRAT_METHODS - keep:
        setattr(trader, m, lambda *a, **k: {})
    # Passive MM fallbacks aren't param-driven; disable for grid runs too.
    trader._run_passive_mm = lambda *a, **k: {}

def run_with_constants(constants_dict, book_cache, ticks, mid_arr, listings,
                      products=PRODUCTS, isolate=None):
    """Apply constants to trader_mod, fresh Trader, run sim, return summary.
    If `isolate` is a category name, only that category's strategies run."""
    saved = {k: getattr(trader_mod, k) for k in constants_dict}
    try:
        for k, v in constants_dict.items():
            setattr(trader_mod, k, v)
        trd = trader_mod.Trader()
        if isolate:
            isolate_category(trd, isolate)
        res = simulate(book_cache, ticks, products, listings, trd, mid_arr,
                      verbose=False, log_errors=True)
    finally:
        for k, v in saved.items():
            setattr(trader_mod, k, v)
    diffs = np.diff(res["cash_history"], prepend=0.0)
    sharpe = diffs.mean()/diffs.std()*np.sqrt(len(diffs)) if diffs.std()>1e-9 else 0.0
    return {"pnl": res["pnl_total"], "sharpe": sharpe, "trades": len(res["trade_log"]),
           "errors": len(res["errors"])}

def grid_search(grid, name="", validate_top=3):
    """Two-phase: rank all configs on TRAIN, re-eval top-`validate_top` on VALID.
    With category isolation this is ~10x faster than the old eval-everything-twice."""
    if not grid:
        return pd.DataFrame()
    keys = list(grid[0].keys())
    train_rows = []
    for cfg in tqdm(grid, desc=f"grid:{name} train"):
        s = run_with_constants(cfg, BOOK_CACHE_TRAIN, TICKS_TRAIN, MID_TRAIN,
                              LISTINGS, isolate=name)
        train_rows.append({**cfg,
                           "train_pnl": s["pnl"], "train_sharpe": s["sharpe"],
                           "train_trades": s["trades"], "train_errors": s["errors"]})
    df = pd.DataFrame(train_rows).sort_values("train_pnl", ascending=False).reset_index(drop=True)

    # Re-evaluate top configs on validation slice
    top = df.head(validate_top).copy()
    v_pnl, v_sharpe, v_trades = [], [], []
    for _, row in tqdm(top.iterrows(), total=len(top), desc=f"grid:{name} valid"):
        cfg = {k: row[k] for k in keys}
        s = run_with_constants(cfg, BOOK_CACHE_VALID, TICKS_VALID, MID_VALID,
                              LISTINGS, isolate=name)
        v_pnl.append(s["pnl"]); v_sharpe.append(s["sharpe"]); v_trades.append(s["trades"])
    top["valid_pnl"] = v_pnl
    top["valid_sharpe"] = v_sharpe
    top["valid_trades"] = v_trades
```



```
# Stitch back: only top rows have valid columns; bottom rows show NaN for clarity
df["valid_pnl"] = np.nan; df["valid_sharpe"] = np.nan; df["valid_trades"] = np.nan
df.loc[top.index, ["valid_pnl", "valid_sharpe", "valid_trades"]] = top[
    ["valid_pnl", "valid_sharpe", "valid_trades"]].values
return df
```

```
# 8.2 – Grids per category (REDUCED for overnight run).
# z_exit and z_stop fixed to sensible defaults; we sweep z_enter and window only.
# Total: 9+9+9+3 = 39 configs.
```

```
GRID_SNACK = [
    {"SNACK_Z_ENTER": ze, "CLASSICS_WINDOW": w, "BERRIES_WINDOW": w}
    for ze in (1.0, 1.5, 2.0)
    for w in (300, 500, 800)
]
print("SNACK grid size:", len(GRID_SNACK))
```

```
GRID_SLEEP = [
    {"SLEEP_Z_ENTER": ze, "SLEEP_WINDOW": w}
    for ze in (1.5, 2.0, 2.5)
    for w in (500, 1000, 1500)
]
print("SLEEP grid size:", len(GRID_SLEEP))
```

```
GRID_ROBOTS = [
    {"ROBOTS_Z_ENTER": ze, "ROBOTS_WINDOW": w}
    for ze in (1.5, 2.0, 2.5)
    for w in (500, 1000, 1500)
]
print("ROBOTS grid size:", len(GRID_ROBOTS))
```

```
GRID_VISORS = [
    {"VISORS_Z_ENTER": ze, "VISORS_WINDOW": w}
    for ze in (1.5, 2.0, 2.5)
    for w in (500, 1000, 1500)
]
print("VISORS grid size:", len(GRID_VISORS))
```

```
GRID_BRONZE = [
    {"BRONZE_Z_ENTER": ze}
    for ze in (2.0, 2.5, 3.0)
]
print("BRONZE grid size:", len(GRID_BRONZE))
```

```
SNACK grid size: 9
SLEEP grid size: 9
ROBOTS grid size: 9
VISORS grid size: 9
BRONZE grid size: 3
```

```
# 8.3 – Run the grid (FAST: ~7 min total with isolation + 2-phase + 0.25 fraction).
# Speedup stack:
# - category isolation (cell 8.1): only target strategies run -> ~5x
# - two-phase: rank on TRAIN (all configs), validate top-3 on VALID -> ~2x vs old
# - GRID_TICK_FRACTION 0.25: cleaner signal under isolation -> ~1.6x
# To go even faster: drop fraction to 0.2 (~5min). To be more thorough: bump to 0.4.
```

```
GRID_TICK_FRACTION = 0.25
```

```
_train_full = TICKS_TRAIN
_valid_full = TICKS_VALID
TICKS_TRAIN = _train_full[: int(len(_train_full) * GRID_TICK_FRACTION)]
TICKS_VALID = _valid_full[: int(len(_valid_full) * GRID_TICK_FRACTION)]
print(f"Grid uses {len(TICKS_TRAIN)} train + {len(TICKS_VALID)} valid ticks (full = {len(_train_full)}+{len(_valid_full)})")
```

```
DO_GRIDS = {
    "snack": GRID_SNACK,
    "sleep": GRID_SLEEP,
    "robots": GRID_ROBOTS,
    "visors": GRID_VISORS,
    "bronze": GRID_BRONZE,
}
```

```
GRID_RESULTS = {}
```

```

try:
    for name, grid in DO_GRIDS.items():
        GRID_RESULTS[name] = grid_search(grid, name=name)
        print(f"\n-- {name}: top configs (sorted by train_pnl, valid_pnl shown for top-3) --")
        print(GRID_RESULTS[name].head(5).to_string(index=False))
finally:
    TICKS_TRAIN = _train_full
    TICKS_VALID = _valid_full
    print("Restored full TICKS_TRAIN/VALID for downstream cells.")

```

Grid uses 5000 train + 2500 valid ticks (full = 20000+10000).

grid:snack train: 89%

8/9 [03:27<00:25, 25.83s/it]

```

# 8.4 – Pick best valid config per category, print recommended diff
print("=" * 70)
print("RECOMMENDED CONSTANT UPDATES (best validation PnL per category):")
print("=" * 70)

current_vals = {
    "SNACK_Z_ENTER": trader_mod.SNACK_Z_ENTER,
    "SNACK_Z_EXIT": trader_mod.SNACK_Z_EXIT,
    "SNACK_Z_STOP": trader_mod.SNACK_Z_STOP,
    "CLASSICS_WINDOW": trader_mod.CLASSICS_WINDOW,
    "BERRIES_WINDOW": trader_mod.BERRIES_WINDOW,
    "SLEEP_Z_ENTER": trader_mod.SLEEP_Z_ENTER,
    "SLEEP_Z_EXIT": trader_mod.SLEEP_Z_EXIT,
    "SLEEP_Z_STOP": trader_mod.SLEEP_Z_STOP,
    "SLEEP_WINDOW": trader_mod.SLEEP_WINDOW,
    "ROBOTS_Z_ENTER": trader_mod.ROBOTS_Z_ENTER,
    "ROBOTS_Z_EXIT": trader_mod.ROBOTS_Z_EXIT,
    "ROBOTS_Z_STOP": trader_mod.ROBOTS_Z_STOP,
    "ROBOTS_WINDOW": trader_mod.ROBOTS_WINDOW,
    "VISORS_Z_ENTER": trader_mod.VISORS_Z_ENTER,
    "VISORS_Z_EXIT": trader_mod.VISORS_Z_EXIT,
    "VISORS_Z_STOP": trader_mod.VISORS_Z_STOP,
    "VISORS_WINDOW": trader_mod.VISORS_WINDOW,
    "BRONZE_Z_ENTER": trader_mod.BRONZE_Z_ENTER,
    "BRONZE_Z_EXIT": trader_mod.BRONZE_Z_EXIT,
    "BRONZE_Z_STOP": trader_mod.BRONZE_Z_STOP,
}

for cat, df in GRID_RESULTS.items():
    best = df.sort_values("valid_pnl", ascending=False).iloc[0]
    print(f"\n[{cat}] best valid_pnl={best['valid_pnl']:.1f} train_pnl={best['train_pnl']:.1f} valid_sharpe={best['valid_sharpe']:.1f}")
    for k in df.columns:
        if k.startswith(("train_", "valid_")): continue
        if k in current_vals:
            cur = current_vals[k]
            new = best[k]
            if cur != new:
                print(f" {k:20s} current={cur} -> suggested={new}")

```

✓ Section 9 – Diagnostics

Per-product position traces, equity curves, trade counts, and suspicious-fill checks (any tick where `position` exceeded the limit; any trader exception).

```

# 9.1 – Trade count per product
import collections
trade_counts = collections.Counter()
for (i, day, ts, sym, side, price, qty) in RESULT_FULL["trade_log"]:
    trade_counts[sym] += 1

tc = pd.Series(trade_counts).sort_values(ascending=False)
print("Top 15 products by trade count:")
print(tc.head(15))
print("\nProducts with zero trades:")
zero_traders = [p for p in PRODUCTS if trade_counts.get(p, 0) == 0]
print(zero_traders)

```

```
# 9.2 – Per-category position trace plots (2x5 grid)
fig, axes = plt.subplots(5, 2, figsize=(13, 14), sharex=True)
axes = axes.flatten()
for ax, (cat, syms) in zip(axes, CATEGORIES.items()):
    for s in syms:
        if s not in RESULT_FULL["position_history"]:
            continue
        ax.plot(RESULT_FULL["position_history"][s], label=s, linewidth=0.7)
    ax.set_title(cat, fontsize=10)
    ax.axhline(10, color="k", linewidth=0.4, linestyle="--")
    ax.axhline(-10, color="k", linewidth=0.4, linestyle="--")
    ax.legend(fontsize=6, loc="best", ncol=2)
    ax.grid(True, alpha=0.3)
plt.tight_layout(); plt.show()
```

```
# 9.3 – Position-limit violation check (should be empty if clipping works)
violations = []
for sym, hist in RESULT_FULL["position_history"].items():
    cap = cap_for(sym)
    over = np.where(np.abs(hist) > cap)[0]
    if len(over):
        violations.append((sym, cap, len(over), int(np.abs(hist[over]).max())))
if violations:
    print("WARNING – position-limit violations:")
    for v in violations:
        print(" ", v)
else:
    print("OK – no position-limit violations.")
```

```
# 9.4 – Trader exceptions (caught by simulate's try/except)
if RESULT_FULL["errors"]:
```